

Animação Física Simplificada em Web

Trabalho de Conclusão de Curso

Diego Vasconcelos Schardosim de Matos

Orientador: Prof. Nome do Orientador

Universidade Federal do Rio de Janeiro
Instituto de Computação

9 de maio de 2026

Sumário

Introdução

Animação Baseada em Física

Detecção de Colisões

Resposta a Colisões

Otimizações

Resultados

Conclusão

Motivação

- ▶ Computação Gráfica combina matemática e arte para representar fenômenos complexos [1]
- ▶ Aplicações interativas exigem realismo visual e alto desempenho (≥ 6 FPS)
- ▶ Animação física une dois domínios:
 - ▶ **Geometria** — detecção e cálculo de contatos
 - ▶ **Física** — evolução dinâmica dos corpos
- ▶ Método de Jakobsen [2]: esquema simplificado sem torques, matrizes de orientação nem tensores de inércia explícitos — mas com lacunas a explorar

Pipeline de animação física

1. Definição dos corpos
2. Integração numérica
3. Detecção de colisões
4. Resposta às colisões
5. Renderização (WebGL)

Motores de referência: Havok, PhysX, Bullet, Box2D — todos inspiram a proposta Web deste trabalho.

Objetivos

Objetivo Geral

Desenvolver um protótipo de animação física 2D para a Web, inspirado no método de Jakobsen, com detecção e resposta a colisões.

Objetivos Específicos:

1. Revisar os principais conceitos de animação baseada em física e integração numérica
2. Implementar algoritmos de detecção e resposta a colisão
3. Desenvolver simulação física baseada em partículas e restrições pelo método de Jakobsen [2]
4. Aplicar técnicas de otimização e processamento *multi-threaded*
5. Avaliar o desempenho e a estabilidade do sistema em diferentes cenários (*benchmarks*)

Conceitos

Keyframe vs. Física

- ▶ **Keyframe:** animador define manualmente posições e rotações ao longo do tempo
- ▶ **Física:** comportamento *emerge* de forças, restrições e interações entre corpos

Ciclo de simulação:

1. Coleta e soma de forças (gravidade, vento, atrito. . .)
2. Integração temporal das equações de movimento
3. Detecção e resposta a colisões
4. Atualização de posições e renderização

Foco: verossimilhança visual

O objetivo **não** é precisão científica, mas **plausibilidade** — transmitir sensação convincente de peso, inércia e colisão ao usuário.

Representação dos Corpos

Corpo Rígido

Forma e volume **invariáveis**

Corpo Deformável

Forma varia sob forças externas.

Método de Integração Numérica

Método de Verlet [2]

Método de segunda ordem que **não armazena velocidade explicitamente**. A nova posição é calculada a partir das posições atual e anterior:

$$\vec{x}_{t+\Delta t} = 2 \vec{x}_t - \vec{x}_{t-\Delta t} + \vec{a}_t \Delta t^2$$

Velocidade derivada implicitamente quando necessária:

$$\vec{v}_t \approx \frac{\vec{x}_{t+\Delta t} - \vec{x}_{t-\Delta t}}{2 \Delta t}$$

Vantagens

- ▶ Estável sob restrições geométricas
- ▶ Menos armazenamento por partícula
- ▶ Método adotado por Jakobsen

Comparação com Euler semi-implícito

- ▶ Euler: $\vec{v}_{t+\Delta t}$ calculada explicitamente
- ▶ Verlet: velocidade embutida na diferença de posições
- ▶ Ambos adequados para tempo real

Restrições Geométricas — Jakobsen

Restrição de distância (linear)

Mantém distância fixa d entre partículas i e j a todo instante:

$$|\vec{x}_i - \vec{x}_j| = d$$

Após cada passo de integração, as partículas são **projetadas** de volta para a configuração válida.

Resolvendo Restrições Concorrentes por Relaxamento

Método iterativo de Jakobsen (Gauss-Seidel)

Restrições são resolvidas **sequencialmente** e repetidas até convergência — geralmente 3 a 5 iterações são suficientes.

Algoritmo de relaxamento para restrição linear:

$$\vec{\delta} = \vec{x}_2 - \vec{x}_1, \quad c = \frac{\|\vec{\delta}\| - d}{\|\vec{\delta}\|}$$

$$\vec{x}_1 \leftarrow \vec{x}_1 - \epsilon \vec{\delta} c, \quad \vec{x}_2 \leftarrow \vec{x}_2 + \epsilon \vec{\delta} c$$

- ▶ $\epsilon = 1$: partículas movidas *imediatamente* para a posição correta (haste rígida)
- ▶ $\epsilon < 1$: convergência gradual (mola rígida)
- ▶ Mais iterações $\Rightarrow$ animação mais rígida visualmente

Detecção de Colisões — Visão Geral

Objetivo

Identificar quando dois objetos entram em contato e fornecer informações geométricas (vetor de penetração, normal) para a etapa de resposta física.

Dois algoritmos estudados e implementados, ambos eficientes para objetos **convexos**:

- ▶ **SAT** — Teorema do Eixo Separador
- ▶ **GJK** — Gilbert-Johnson-Keerthi

Teorema do Eixo Separador (SAT)

Teorema

Dois polígonos convexos A e B **não se intersectam** se, e somente se, existe um hiperplano separador P tal que A está de um lado de P e B está do outro [?].

Ideia prática: testar as normais das arestas de ambos os polígonos como eixos candidatos. Para cada eixo $\hat{n}$:

$$\text{Sobreposição} \iff \max_A \geq \min_B \wedge \max_B \geq \min_A$$

Se *qualquer* eixo separar as projeções, os objetos **não colidem** e o algoritmo encerra imediatamente.

[figura: eixo separador]

Algoritmo GJK — Fundamentos

Ideia central

Dois objetos A e B se intersectam se, e somente se, a **diferença de Minkowski** (CSO) contém a origem:

$$A - B = \{ \vec{x} - \vec{y} \mid \vec{x} \in A, \vec{y} \in B \}$$

$$d(A, B) = \min_{c \in \text{CSO}} \| c \|$$

O CSO **não é calculado explicitamente**. Em vez disso, amostras são obtidas sob demanda pela **função suporte**:

$$S_{A-B}(\vec{d}) = S_A(\vec{d}) - S_B(-\vec{d})$$

[figura: diferença de Minkowski]

Algoritmo GJK — Iteração sobre Simples

O GJK constrói iterativamente **simplices** dentro do CSO, aproximando-se da origem:

- ▶ 0-simplex: ponto
- ▶ 1-simplex: segmento
- ▶ 2-simplex: triângulo (em 2D)

A cada iteração, um novo ponto de suporte $\vec{s}$ é adicionado. Se $\vec{s} \cdot \vec{d} \leq 0$, a origem **não pertence** ao CSO — sem colisão.

[figura: 0, 1, 2-simplex]

O procedimento **NearestSimplex** reduz o simplex ao subconjunto mínimo que

Subalgoritmo de Ericson — Caso 1-simplex

Segmento RS divide o espaço em **três regiões**: região do vértice R , região do vértice S e região da aresta $\vec{rs}$.

- ▶ Origem na **região de R** : descarta S , $\vec{d} = \vec{r\vec{o}}$
- ▶ Origem na **região de S** : descarta R , $\vec{d} = \vec{s\vec{o}}$
- ▶ Origem na **região da aresta**: mantém $\{R, S\}$,
 $\vec{d} = (\vec{rs} \times \vec{r\vec{o}}) \times \vec{rs}$

Regiões de vértice estão além do limite do CSO $\Rightarrow$ origem nessas regiões implica **sem colisão**.

[figura: regiões do 1-simplex]

Subalgoritmo de Ericson — Caso 2-simplex

Triângulo RST possui **7 regiões**, mas o histórico de construção elimina 4 delas. Verifica-se apenas: aresta $\vec{rs}$, aresta $\vec{rt}$ e interior do $\triangle RST$.

Seja $\vec{n} = \vec{rs} \times \vec{rt}$:

- ▶ Se $(\vec{n} \times \vec{rs}) \cdot \vec{ro} > 0 \Rightarrow$ origem na região de $\vec{rs}$: executar caso 1-simplex($\{R, S\}$)
- ▶ Se $(\vec{rt} \times \vec{n}) \cdot \vec{ro} > 0 \Rightarrow$ origem na região de $\vec{rt}$: executar caso 1-simplex($\{R, T\}$)
- ▶ Caso contrário $\Rightarrow$ origem no interior do triângulo: **colisão detectada**

[figura: regiões do 2-simplex]

Resposta a Colisões — Conceitos

Duas operações principais

1. **Correção de posição:** eliminar a interpenetração
2. **Correção de velocidade:** evitar novo contato imediato

Principais abordagens na literatura:

- ▶ **Método do Impulso** — aplica impulsos instantâneos que alteram velocidades preservando momento linear e angular (Havok, Bullet)
- ▶ **Método de Penalidades** — forças de repulsão proporcionais à penetração (modelo mola-amortecimento); simples, mas sensível a parâmetros de rigidez
- ▶ **Restrições + Lagrange** — formulação robusta; exige solução de sistemas lineares — oneroso em plataformas Web
- ▶ **Jakobsen (este trabalho)** — colisões tratadas como **restrições geométricas** corrigidas por posição, dentro do laço de relaxamento

Pontos de Contato e Manifold de Colisão

Pontos de contato

Posições onde dois objetos se tocam.

Obtidos a partir da geometria no instante em que se detecta a interpenetração.

Podem ser um único ponto (vértice-face) ou múltiplos (face-face).

Manifold de colisão

Estrutura que agrega os pontos de contato com:

- ▶ **Normal de colisão** $\hat{n}$ — direção de separação entre os objetos
- ▶ **Profundidade de penetração** d por ponto

[figura: manifold de colisão]

SAT com Cálculo do MTV

Vetor de Translação Mínima (MTV)

Extensão do SAT: em vez de encerrar ao encontrar um eixo separador, **todos os eixos são testados**. O eixo com a **menor sobreposição** fornece a normal de contato $\hat{n}$ e a profundidade de penetração δ .

Para cada aresta normal $\hat{n}$ de A e B :

Δ = sobreposição das projeções de A e B

- ▶ Se $\Delta \leq \epsilon \Rightarrow$ **sem colisão**, retorna imediatamente
- ▶ Se $\Delta < \delta_{\min} \Rightarrow$ atualiza MTV:
 $\delta \leftarrow \Delta, \hat{n}_{\text{MTV}} \leftarrow \hat{n}$

Ao fim, $(\hat{n}_{\text{MTV}}, \delta)$ descrevem a menor



Algoritmo de Expansão de Politopos (EPA)

Motivação

O GJK detecta a colisão, mas não fornece $\hat{n}$ nem δ . O EPA resolve o problema dual: encontrar o **MTV** a partir do simplex final do GJK.

Iteração:

1. Selecionar a **aresta mais próxima** da origem
2. $\vec{p}$: ponto mais próximo da origem na aresta ($\|\vec{p}\| = \text{limite inferior de } \delta$)
3. $\vec{w} = S_{\text{CSO}}(\vec{p})$: novo vértice na fronteira do CSO ($\|\vec{w}\| = \text{limite superior de } \delta$)

[figura: iterações do EPA]

Cálculo dos Pontos de Contato

Após obter $\hat{n}$ e δ (via SAT+MTV ou EPA), identificam-se as arestas e vértices envolvidos por um **método adaptado de recorte**:

1. Para cada objeto, selecionar o **vértice mais distante** ao longo de $\hat{n}$; verificar os dois adjacentes e escolher a aresta que faz o maior ângulo com $\hat{n}$
2. A aresta **mais perpendicular** a $\hat{n}$ torna-se a **aresta de referência**; a outra é a **aresta incidente**
3. Vértices da aresta de referência + vértice incidente com maior produto escalar com $-\hat{n}$ formam o manifold

[figura: arestas envolvidas]

[figura: aresta ref. e incidente]

Processo de Separação — Jakobsen

Correção total: $\vec{c} = d \cdot \hat{n}$, distribuída pelas massas inversas:

$$\vec{c}_A = -\frac{w_B}{w_A + w_B} \vec{c}, \quad \vec{c}_B = +\frac{w_A}{w_A + w_B} \vec{c}$$

Caso vértice–aresta (vértice $p \in A$, aresta $\overline{rs} \in B$):

$$\alpha = \frac{(p - r) \cdot (s - r)}{\|s - r\|^2}, \quad w = r + \alpha(s - r)$$

$$p' = p + \vec{c}_A$$

$$r' = r + (1 - \alpha) \vec{c}_B, \quad s' = s + \alpha \vec{c}_B$$

O vértice mais próximo de w recebe maior fração da correção, preservando a coerência

[figura: separação
vértice-aresta]

Resposta a Colisões — Projeção de Posição

Dado contato com normal $\hat{\mathbf{n}}$ e penetração d :

$$\vec{\delta} = \hat{\mathbf{n}} \cdot \frac{\max(d - \epsilon, 0)}{m_A^{-1} + m_B^{-1}}$$

Correção das posições:

$$\vec{x}_A += m_A^{-1} \vec{\delta}, \quad \vec{x}_B -= m_B^{-1} \vec{\delta}$$

Consistência com o método de Jakobsen

A resposta a colisões é tratada como uma **restrição de distância mínima** entre os corpos, projetando as posições diretamente — sem necessidade de calcular impulsos explícitos.

Otimizações — O Problema

Complexidade ingênua

Com n objetos, o número de pares a testar é:

$$\frac{n(n-1)}{2} = O(n^2)$$

Mesmo para valores moderados de n , esse custo torna-se impraticável em tempo real.

Solução: separar a detecção em duas fases:

1. **Filtragem (*Broad Phase*)** — descartar rapidamente pares impossíveis com testes baratos e conservadores
2. **Refinamento (*Narrow Phase*)** — aplicar SAT / GJK / EPA apenas nos pares candidatos restantes

A premissa fundamental: objetos só podem colidir se estiverem **suficientemente próximos**.

Redução esperada

Em cenários bem comportados, a filtragem reduz a complexidade para $O(n)$ ou $O(n \log n)$, embora no pior caso (todos concentrados em poucas células) possa regredir a $O(n^2)$.

Objetos Delimitadores

Substituem a geometria original por formas mais simples para testes rápidos de sobreposição. Propriedade fundamental:

- ▶ Se os delimitadores **não** se intersectam $\Rightarrow$ os objetos **definitivamente não** colidem
- ▶ Se se intersectam $\Rightarrow$ *possível* colisão (falso positivo admissível)

Filtragem com Grade Uniforme

O espaço é subdividido em células de tamanho fixo C . Cada objeto é associado às células interceptadas pela sua AABB:

$$i = \left\lfloor \frac{x}{C} \right\rfloor, \quad j = \left\lfloor \frac{y}{C} \right\rfloor$$

Chave linear da célula:

$$Key_{\text{cell}} = i + j \times n_{\text{col}}$$

Os pares $(Key_{\text{cell}}, Index_{\text{obj}})$ são armazenados em um array e **ordenados** pela chave. Objetos na mesma célula ficam contíguos — favorece localidade de cache.

Dois objetos são **candidatos** somente se compartilharem ao menos uma célula.

Filtragem — Deduplicação e Escolha de C

Eliminação de duplicatas ao percorrer o array ordenado:

1. Par (A, B) testado por ordem — nunca repetir (B, A)
2. Objeto grande interceptando múltiplas células pode gerar o mesmo par em células distintas — testar uma única vez

Escolha do tamanho de célula C :

- ▶ Idealmente cada objeto ocupa ≈ 1 célula (máx. 4 em 2D)
- ▶ C pequeno demais: custo de atualização elevado
- ▶ C grande demais: muitos objetos por célula, filtragem ineficaz
- ▶ Grande variação de escala: considerar abordagens hierárquicas

Estratégia de reconstrução

A grade é **reconstruída a cada passo de tempo** a partir de um array linear com contador zerado. Trade-off deliberado: simplicidade e previsibilidade em vez de atualizações incrementais.

Narrow Phase

Após a filtragem, os pares candidatos passam ao refinamento via **SAT** ou **GJK+EPA** para confirmação precisa da colisão.

Metodologia de Testes

Quatro configurações avaliadas, variando o número de objetos de 100 a 1000 (incrementos de 100), com 180 quadros por configuração:

1. SAT **sem** fase de filtragem
 2. SAT **com** grade uniforme na filtragem
 3. GJK/EPA **sem** fase de filtragem
 4. GJK/EPA **com** grade uniforme na filtragem
- ▶ Testes executados **sem visualização** — isola o custo computacional da detecção de colisão
 - ▶ Distribuição inicial por *Poisson Disk Sampling* — amostras uniformes com distância mínima garantida
 - ▶ Semente pseudoaleatória fixa para reprodutibilidade
 - ▶ Métricas: tempo médio de execução, número de testes de colisão e tempo por etapa da simulação

Impacto da Filtragem — Número de Testes de Colisão

[figura: número médio de testes de colisão (escala log)]

Tempo por Etapa — Modo SAT

[figura: tempo relativo por etapa — modo SAT]

Tempo por Etapa — Modo GJK/EPA

[figura: tempo relativo por etapa — modo GJK/EPA]

Análise: Gargalo do Relaxamento

Por que o relaxamento domina?

O método de Jakobsen exige **múltiplas iterações** (5 neste trabalho) para convergir. O custo cresce proporcionalmente ao número de restrições e passos de relaxamento.

Caminhos para otimização identificados:

- ▶ Reduzir o número de iterações de relaxamento
- ▶ Substituir a grade uniforme por estruturas hierárquicas (BVH, *Quadtree*) na filtragem
- ▶ Paralelizar etapas via **multi-thread** (Web Workers)
- ▶ Delegar cálculos intensivos à GPU via **WebGPU**

Comparação com *Planck.js* — Tempo de Execução

[figura: tempo médio de execução — nossa impl. vs Planck.js]

Por que *Planck.js* é mais rápida?

Nossa implementação

- ▶ Resolução por **relaxamento iterativo** (Jakobsen) — múltiplas iterações por passo de tempo
- ▶ Filtragem via **grade uniforme** — simples, mas menos adaptativa a variações de escala

Planck.js (baseada no Box2D)

- ▶ Resolução por **impulsos** — colisões resolvidas diretamente no nível dos corpos rígidos, mais eficiente por passo de tempo
- ▶ Filtragem com **árvore dinâmica de AABB** — hierárquica, mais adaptada a cenários heterogêneos

Conclusões

- ▶ Foi implementado um motor de animação física funcional inteiramente em JavaScript/WebGL
- ▶ A integração de Verlet mostrou-se adequada e estável para simulações em tempo real no navegador
- ▶ O método de Jakobsen por relaxamento de restrições proporcionou comportamento visualmente plausível
- ▶ Desempenho satisfatório para aplicações interativas típicas (≤ 300 corpos a 60 FPS)
- ▶ Código aberto e bem documentado — adequado para uso didático

Trabalhos Futuros

1. Implementar grade espacial / BVH para fase ampla mais eficiente
2. Adicionar suporte a articulações e restrições angulares
3. Explorar WebGPU para cálculos físicos em GPU
4. Estender para 3D com suporte a quatérnios

Referências

-  E. Azevedo, A. Conci. *Computação Gráfica*. Campus, 2003.
-  T. Jakobsen. Advanced character physics. *Game Developer Conference*, 2001.
-  C. Ericson. *Real-Time Collision Detection*. Morgan Kaufmann, 2004.
-  E. Catto. *Box2D — A 2D Physics Engine for Games*. <https://box2d.org/>, 2007.
-  L. Brewer. *Matter.js — 2D Physics Engine for the Web*. <https://brm.io/matter-js/>, 2014.

Obrigado!

Dúvidas e discussão

`diegovsm@ic.ufjr.br`